

Task 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if(pid == 0)
    {
        printf("Child process listing current directory:\n");

        execl("/bin/ls", "ls", "-l", NULL);

        printf("Exec failed\n");
        exit(1);
    }
    else
    {
        wait(NULL);
        printf("Parent: Child terminated.\n");
    }

    return 0;
}
```

Output:

Child process listing current directory:

total 28

-rwxr-xr-x. 1 60000 60000 48 Apr 24 19:34 _compile.sh

-rwxr-xr-x. 1 60000 60000 16128 Apr 24 19:34 main

-rw-r--r--. 1 60000 60000 522 Apr 24 19:34 NewFile1.c

-rwxr-xr-x. 1 60000 60000 31 Apr 24 19:34 _run.sh

Parent: Child terminated.

Task 2:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main()
```

```
{
```

```
    printf("Original Process PID: %d\n", getpid());
```

```
    pid_t pid = fork();
```

```
    if(pid < 0)
```

```
    {
```

```
        printf("Fork failed\n");
```

```
        return 1;
```

```
    }
```

```
    if(pid == 0)
```

```
    {
```

```
        printf("\nI am Child Process\n");
```

```
        printf("Child PID: %d\n", getpid());
```

```
        printf("Parent PID: %d\n", getppid());
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("\nI am Parent Process\n");
```

```
        printf("Parent PID: %d\n", getpid());
```

```
        printf("Child PID: %d\n", pid);
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

Original Process PID: 3

I am Parent Process

I am Child Process

Child PID: 4

Parent PID: 3

Parent PID: 3

Child PID: 4

Task 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 1024
```

```
int main(int argc, char *argv[])
{
    if(argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    int source = open(argv[1], O_RDONLY);

    if(source < 0)
    {
        printf("Error opening source file\n");
        return 1;
    }

    int destination = open(argv[2],
        O_WRONLY | O_CREAT | O_TRUNC,
```

```

        0644);

if(destination < 0)
{
    printf("Error opening destination file\n");
    close(source);
    return 1;
}

char buffer[BUFFER_SIZE];
int bytesRead;

while((bytesRead = read(source, buffer, BUFFER_SIZE)) > 0)
{
    if(write(destination, buffer, bytesRead) != bytesRead)
    {
        printf("Write error\n");
        close(source);
        close(destination);
        return 1;
    }
}

if(bytesRead < 0)
{
    printf("Read error\n");
}

close(source);
close(destination);

printf("File copied successfully\n");

return 0;
}

```

Task 4:

```

#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>

```

```

void listDirectory(char path[])
{

```

```

DIR *dir;
struct dirent *entry;
struct stat info;

dir = opendir(path);

if(dir == NULL)
{
    printf("Cannot open directory %s\n", path);
    return;
}

while((entry = readdir(dir)) != NULL)
{
    if(strcmp(entry->d_name, ".") == 0 ||
        strcmp(entry->d_name, "..") == 0)
        continue;

    char fullPath[1000];

    sprintf(fullPath, "%s/%s", path, entry->d_name);

    if(stat(fullPath, &info) == 0)
    {
        if(S_ISDIR(info.st_mode))
        {
            printf("[Directory] %s\n", fullPath);

            listDirectory(fullPath);
        }
        else
        {
            printf("[File] %s\n", fullPath);
        }
    }
}

closedir(dir);
}

int main()
{
    printf("Recursive Directory Listing:\n");
    listDirectory(".");
}

```

```
    return 0;  
}
```

Output:

Recursive Directory Listing:

[File] ./NewFile1.c

[File] ./_compile.sh

[File] ./main

[File] ./_run.sh